Supervised learning

Artificial Intelligence
CSC 480
Rodrigo Canaan
rcanaan@calpoly.edu

Adapted with permission from
Julian Togelius
julian.togelius@nyu.edu

Types of learning

- **Supervised learning**
  Learning to predict or classify labels based on labeled input data

- **Unsupervised learning**
  Finding patterns in unlabeled data

- **Reinforcement learning**
  Learning well-performing behavior from state observations and rewards

Types of learning

- **Supervised learning**
  Learning to predict or classify labels based on labeled input data

- **Unsupervised learning**
  Finding patterns in unlabeled data

- **Reinforcement learning**
  Learning well-performing behavior from state observations and rewards

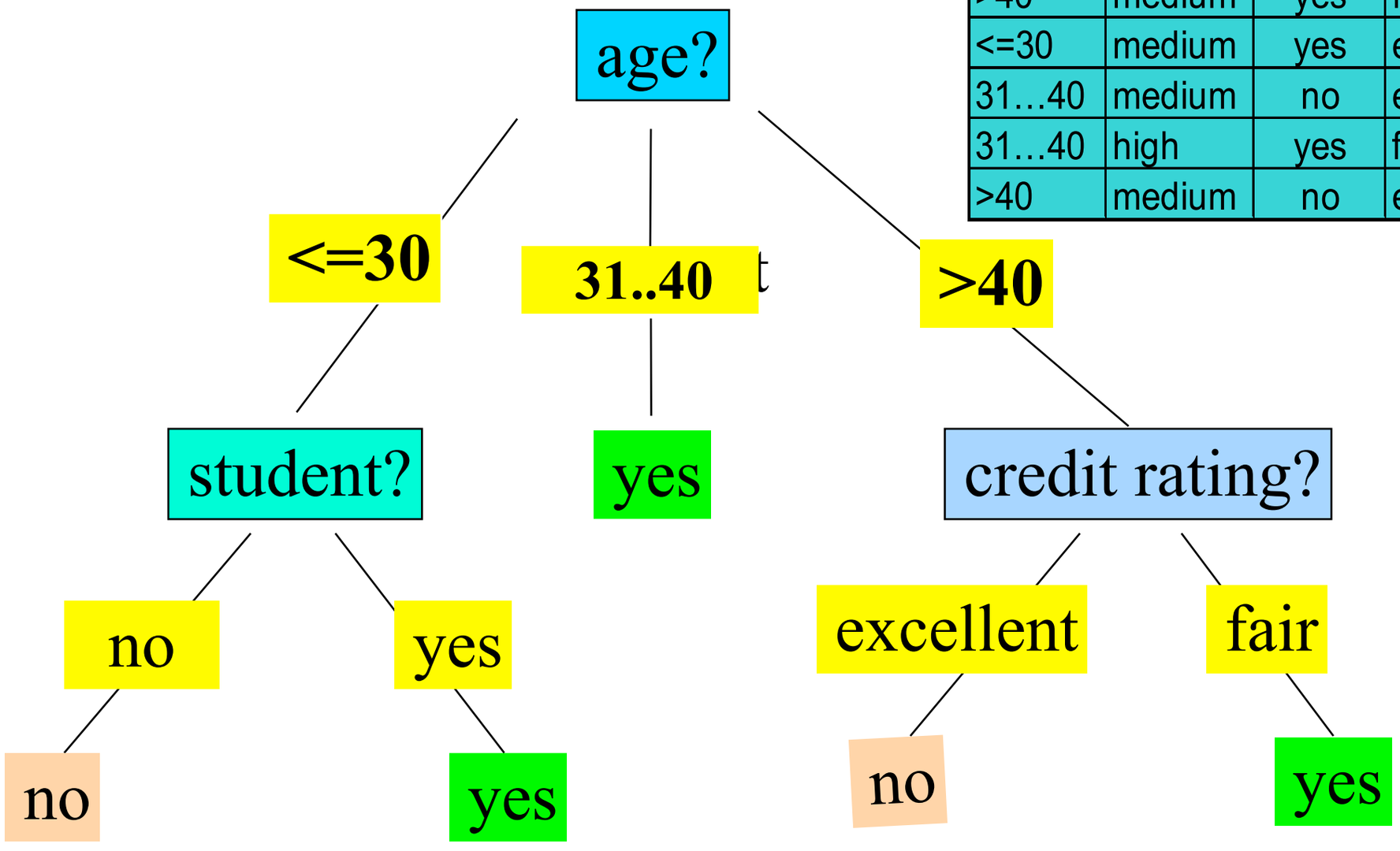
Decision trees

- A popular representation for classifiers
- Human-readable
- Can be learned (induced) efficiently using algorithms based on information theory
  - An eager learning method
- Often yields high-accuracy classifiers

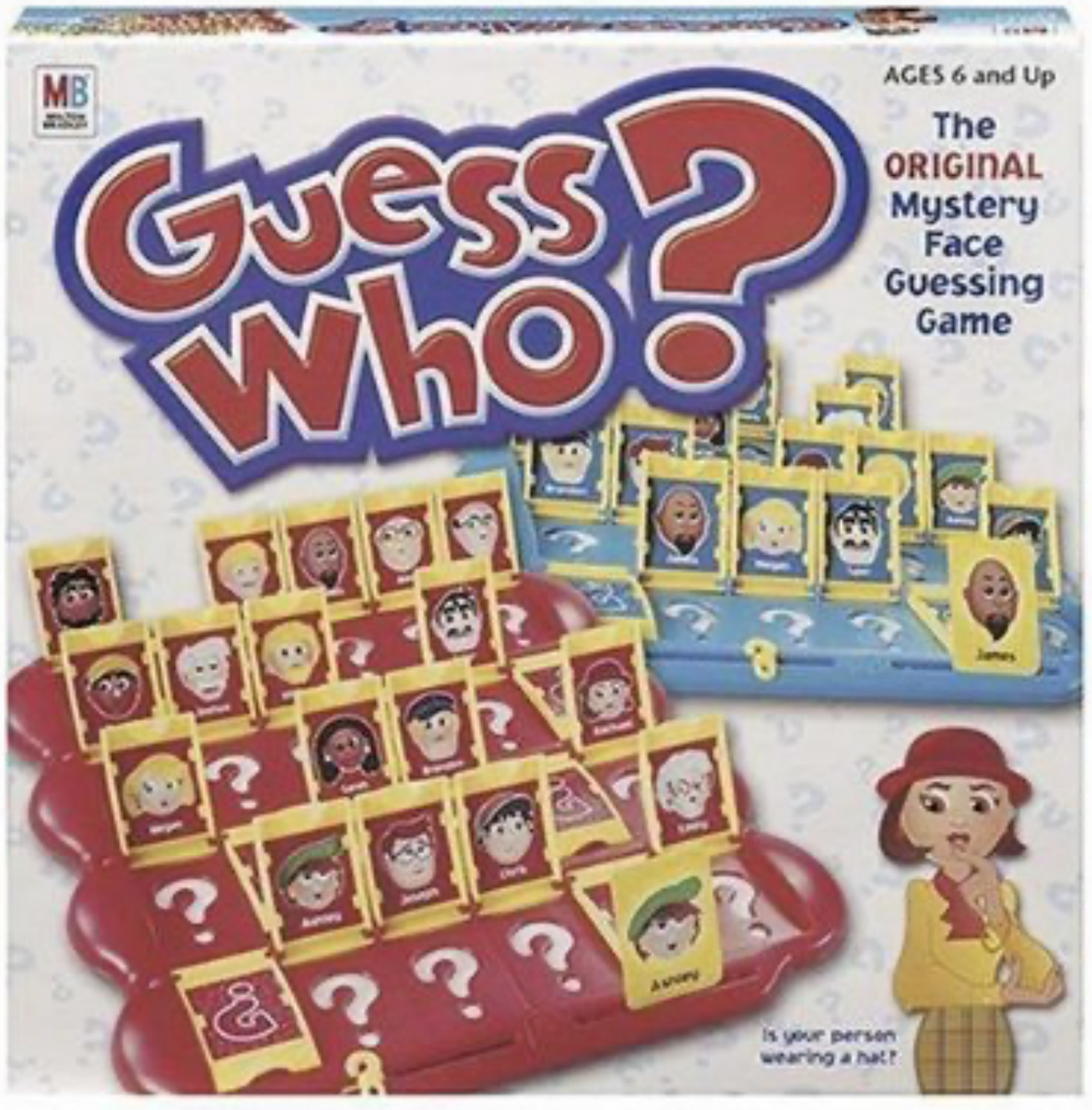
An example

Classify: buys_computer

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no



Analogy: “Guess Who?” Game



The main intuition

- Let's assume all features $X_1, X_2, X_3 \ldots$ and the label Y are binary for now. Our dataset is called D

- Suppose Y occurs on 30% of our data.

- Asking "is X_1 true or false?" splits our data in two

  - D_{X_1}, containing all entries where X_1 is true

  - $D_{\overline{X_1}}$, containing all entries where X_1 is false

The main intuition

- A few things can happen after splitting this way (in our training data)

  A. D_{X_1} contains all Y-positive entries $D_{\overline{X_1}}$ contains all Y-negative ones.

    - This is the dream, assuming real life data matches this pattern

  B. Y is positive about 30% of the time in both D_{X_1} and $D_{\overline{X_1}}$.

    - Asking about X_1 wasn't very helpful

  C. D_{X_1} contains almost all Y-positives mixed with a bunch of Y-negative ones, but $D_{\overline{X_1}}$ is almost entirely Y-negative.

    - We can now make a confident prediction in $D_{\overline{X_1}}$, and keep asking questions about other X_i features in D_{X_1}

The main intuition

- A good feature is one that either results in “pure” splits or that concentrates most of the impurity in one side of the split
- We need a way to quantify purity
- We also need to weight by the prevalence of feature X_1.
  - If X_1 is very rare, finding that the few D_{X_1} entries are very pure isn’t as helpful, as it leaves us with $D_{\overline{X_1}}$ almost as impure as we started with, and with almost all the entries

ID3 Algorithm for Decision Tree Induction

- Tree is constructed in a top-down recursive divide-and-conquer manner

- At start, all the training examples are at the root

- Attributes are categorical (if continuous-valued, they are discretized in advance)

- Examples are partitioned recursively based on selected attributes

  - One possible scenario is we ask first about X1, and if X1 i true, we ask about X2, if X1 is false we ask about X3 instead

- Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain)

ID3 Algorithm for Decision Tree Induction

- Stop partitioning when...
  - **All samples for a given node belong to the same class, or…**
  - There are no remaining attributes for further partitioning – (vote on the leaf) or…
  - There are no samples left
  - Some predetermined number of features have already been used (to avoid overfitting)
  - **For a particular branch, no further feature results in better accuracy in our validation data.**

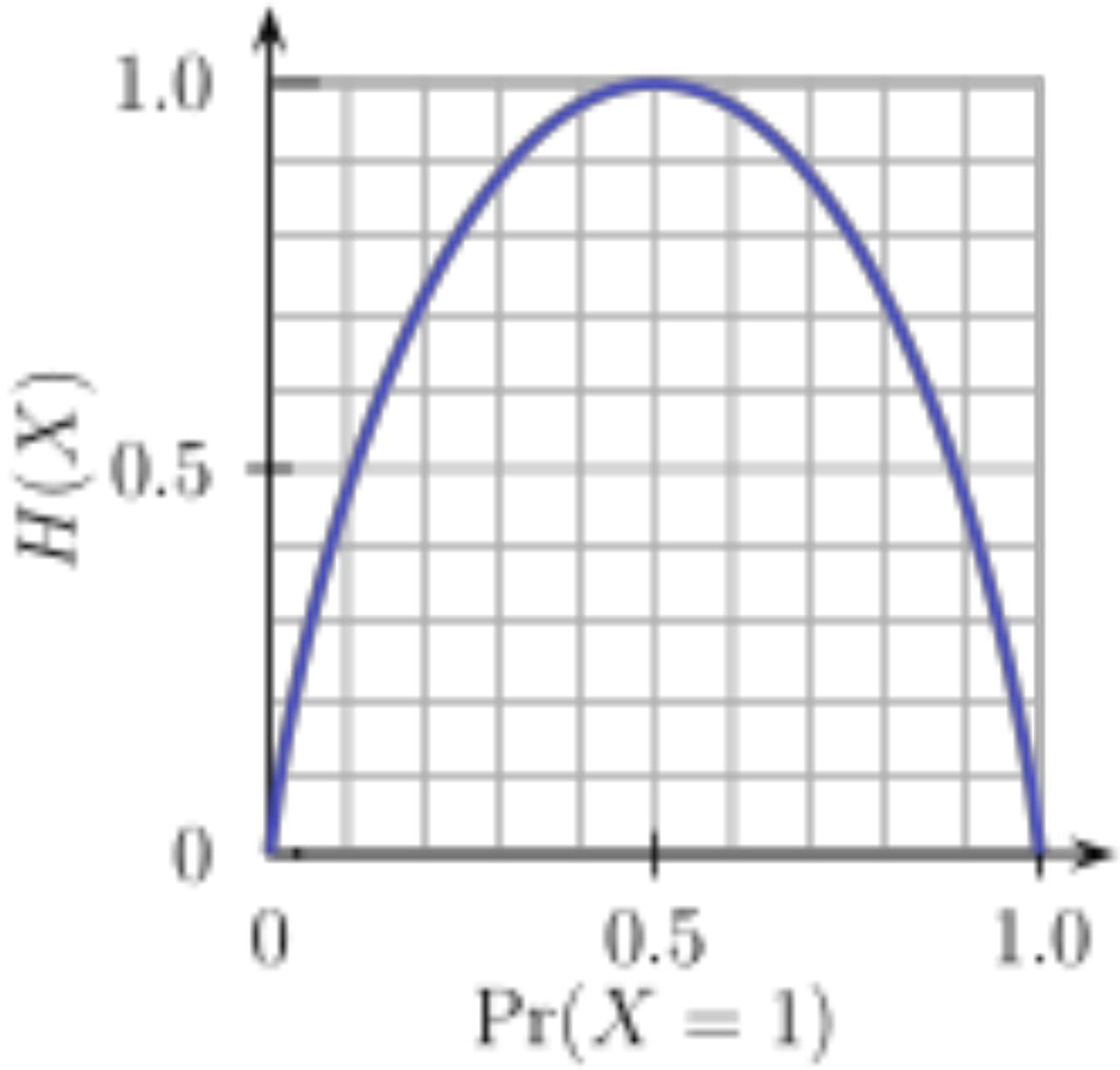
Attribute Selection Measure: Information Gain (ID3/C4.5)

- Assumptions:
  - Our features are $A, B, C, \ldots$ etc, with $v_A, v_B, v_C, \ldots$ possible values in each feature's domain
  - Our label is Y with m possible values $y_1, \ldots, y_m$, indexed by i
- Let p_i be the probability that an arbitrary tuple in D belongs to class y_i, $P(Y = y_i)$ estimated by $|D_{Y=y_i}|/|D||$
- Expected information (entropy) needed to classify a tuple in$D$:

$$Info(D) = -\sum_{i=1}^{m} p_i * log_2(p_i)$$ over target classes (possible labels) i

- Can be interpreted as the number of bits of information needed for classification

Binary entropy function



Attribute Selection Measure: Information Gain (ID3/C4.5)

- Information needed (after using A to split D into v_a partitions $D_{A=j}$) to classify D

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_{A=j}|}{|D|} Info(D_{A=j})$$

-

Attribute Selection Measure: Information Gain (ID3/C4.5)

- Information needed (after using A to split D into v_a partitions $D_{A=j}$) to classify D

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_{A=j}|}{|D|} Info(D_{A=j})$$

- In other words, it's the sum of information of each subset of D filtered by $A = j$,

Attribute Selection Measure: Information Gain (ID3/C4.5)

- Information needed (after using A to split D into v_a partitions $D_{A=j}$) to classify D

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_{A=j}|}{|D|} \times Info(D_{A=j})$$

- In other words, it's the sum of information of each subset of D restricted by $A = j$, weighted by the relative size of each subset

Attribute Selection Measure: Information Gain (ID3/C4.5)

- Information needed (after using A to split D into v_a partitions $D_{A=j}$) to classify D

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_{A=j}|}{|D|} Info(D_{A=j})$$

- Information gained by branching on attribute A

$$Gain(A, D) = Info(D) - Info_A(D)$$

- We want to select the attribute with highest gain (biggest *decrease* in entropy)

Attribute Selection: Information Gain

Class P: buys_computer = “yes”

Class N: buys_computer = “no”

$$Info(D) = I(9,5) = -\frac{9}{14}\log_2(\frac{9}{14}) - \frac{5}{14}\log_2(\frac{5}{14}) = 0.940$$

age	p_i	n_i	$I(p_i, n_i)$
<=30	2	3	0.971
31…40	4	0	0
>40	3	2	0.971

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

$$Info_{age}(D) = \frac{5}{14}I(2,3) + \frac{4}{14}I(4,0) + \frac{5}{14}I(3,2) = 0.694$$

means “age <=30” has 5 out of 14 samples, with 2 yes’es and 3 no’s. Hence

$$Gain(age) = Info(D) - Info_{age}(D) = 0.246$$

Similarly,

$$Gain(income) = 0.029$$

$$Gain(student) = 0.151$$

$$Gain(credit_rating) = 0.048$$

- Input: A data set, S
  Output: A decision tree

    - If all the instances have the same label for the target attribute then return a decision tree that is simply this label

- Else

    - Compute Gain values (see above) for all attributes and select an attribute with the highest value and create a node for that attribute.

    - Make a branch from this node for every value of the attribute

    - Follow each branch by partitioning the dataset to be only instances whereby the value of the branch is present and then go back to 1.

function DECISION-TREE-LEARNING($examples$, $attributes$, $parent_examples$) **returns** tree

  if $examples$ is empty **then return** PLURALITY-VALUE($parent_examples$)
  else if all $examples$ have the same classification **then return** the classification
  else if $attributes$ is empty **then return** PLURALITY-VALUE($examples$)
  else
  $A \leftarrow \text{argmax}_{a \in attributes}$ IMPORTANCE(a, $examples$)
  $tree \leftarrow$ a new decision tree with root test A
  for each value v_k of A **do**
    $exs \leftarrow \{e : e \in examples \textbf{ and } e.A = v_k\}$
    $subtree \leftarrow$ DECISION-TREE-LEARNING(exs, $attributes - A$, $examples$)
    add a branch to $tree$ with label $(A = v_k)$ and subtree $subtree$
  return $tree$

Extending decision trees

- **Regression Trees**

  - The “vote”: If prediction target is real-valued, each leaf node returns average value of datapoints (instead of majority)

  - The “goodness”: minimize error (typically mean squared error) of the node (instead of entropy)

  - The tree edges: use thresholds on the feature value for real-valued features

Extending decision trees

- **Decision Forests (e.g. Random Forests)**
    - Train an ensemble of independent, diverse decision trees.
        - Diversity can be achieved by training each tree on a random subset of data points and/or features
- Inference: Each tree gets a “vote” - take the average or mode of each tree’s prediction, often weighted by their qu
- Each tree can overfit, but the ensemble tends to minimize overfitting in the joint prediction

Extending decision trees

- **Boosting trees**

  - Another type of ensemble method

  - Each successive tree tries to predict correct the the error (loss) of the previous ensemble of trees

  - This can be done by changing weights of misclassified samples (e.g. **AdaBoost** with trees) or using the gradient of the residual loss (gradient boosting e.g. **XGBoost**)